

COMP1117 Tutorial 2

AI USE AND TESTING
AI-GENERATED CODE

Learning outcomes

- ▶ Use AI to question and clarify a programming requirement.
- ▶ Review requirements.md and explain the decisions yourself.
- ▶ Choose tests with expected answers before generating code.
- ▶ Use execution evidence to repair and retest the program.

Our 50-minute tutorial

Time	Activity
0–8 min	AI use, HKU access and VS Code Chat
8–18 min	Workspace and requirements demonstration
18–28 min	Test groups, boundaries and branch coverage
28–40 min	Implement, run tests and investigate failures
40–47 min	Independent check: a changed requirement
47–50 min	Exit questions and evidence checklist

Follow the demonstration and explain the decisions as we go.

Constructive ways to use AI

Learning purpose	A useful request
Clarify a task	What is ambiguous in these requirements?
Compare decisions	Give two options and explain their consequences.
Check understanding	Ask me to predict a result and explain my reasoning.
Review evidence	Which requirement does this failing test contradict?

Do not paste the whole assignment into Chat and ask AI to “Solve it for me”.

Your decisions still matter

- ▶ AI may fill in a missing rule without asking.
- ▶ You confirm the rules with the task brief or tutor.
- ▶ You work out expected outputs independently.
- ▶ You accept an edit only when you can explain it.

The workflow for this session

Stage	Evidence you keep
Understand and agree	Reviewed requirements.md
Prepare tests	Inputs, expected results and a reason for each case
Implement with AI	Code and a record of the changes you accepted
Run, repair and retest	Actual outputs and a short explanation of failures

Implementation starts after the requirements and tests are reviewed.

AI use and testing in assignments

- ▶ **AI use and test-case verification count towards your assignment grade.**
- ▶ Keep a record of your prompts and changes to AI-generated code.
- ▶ Document test inputs, expected outputs and actual results.
- ▶ Explain failures and verify the program again after fixes.

Follow the assignment rubric for the required evidence and marking details.

HKU AI access for this tutorial

[HKU GenAI: genai.hku.hk](https://genai.hku.hk)

- ▶ Sign in with your HKU Portal account.
- ▶ Ask for explanations, examples and feedback to help you learn.
- ▶ Students receive 500,000 AI Credits each month.

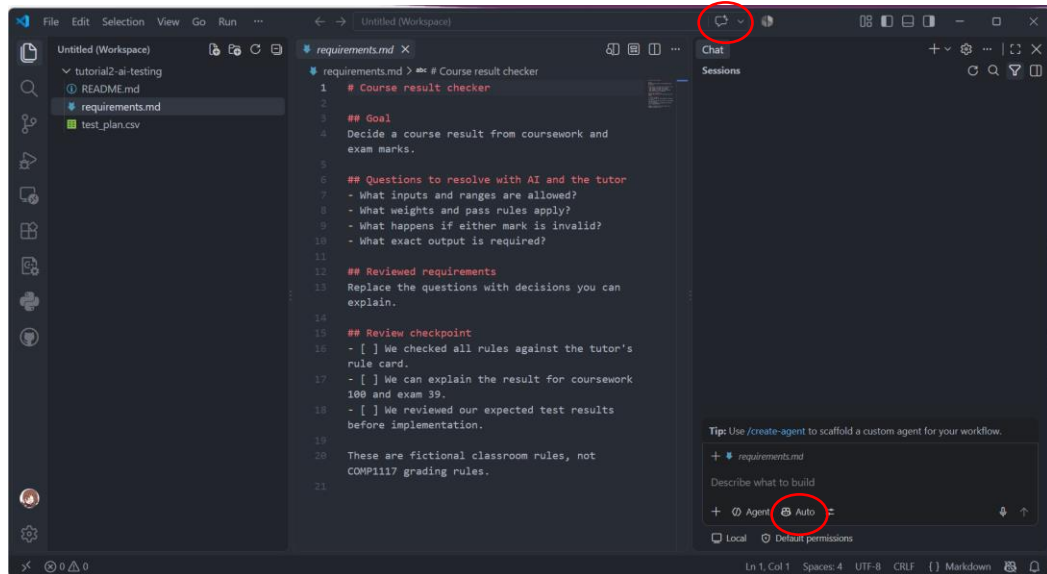
Use GenAI to support your learning.

Do not use this chat to complete your assignments.

[HKU ITS service details](#)

Chat inside VS Code

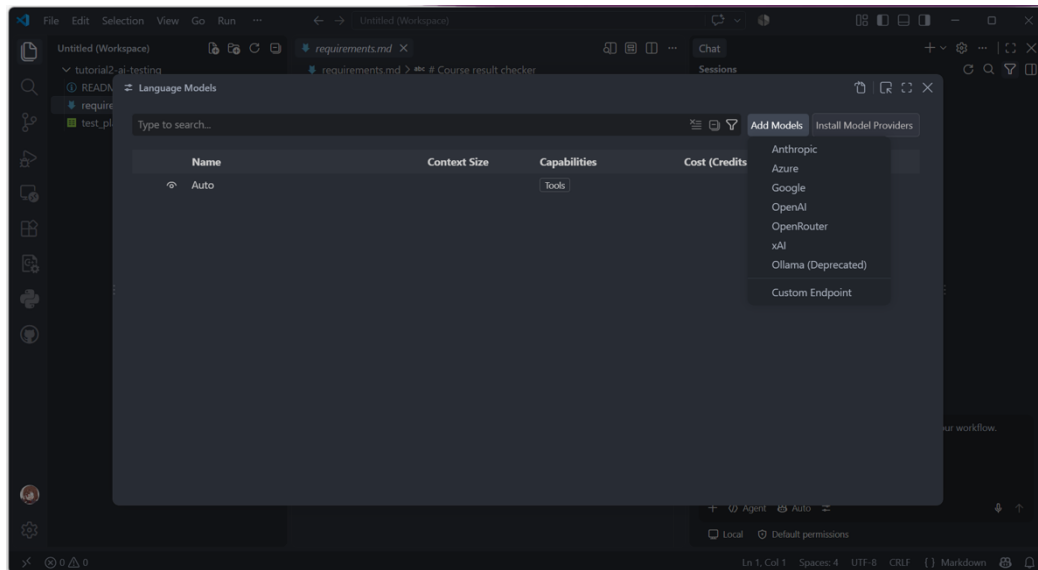
- ▶ Sign in with your GitHub account.
- ▶ Open the built-in Chat view.
- ▶ Type a question and send it.



Keep the discussion beside the files you are working on.

Choosing a chat model

- ▶ Open the model picker in Chat.
- ▶ Select an available model.
- ▶ Repeat one question and compare the explanation.



Available choices depend on your account, location and service settings.

AI agents and your review

An agent can choose actions, edit files and run tools, then use the results to decide what to do next.

Stage	How we use the AI
Requirements discussion	Ask questions and review proposed wording.
Implementation	After review, let Copilot edit and run code.

You decide the expected behaviour and review the evidence.

Our project: a course result checker

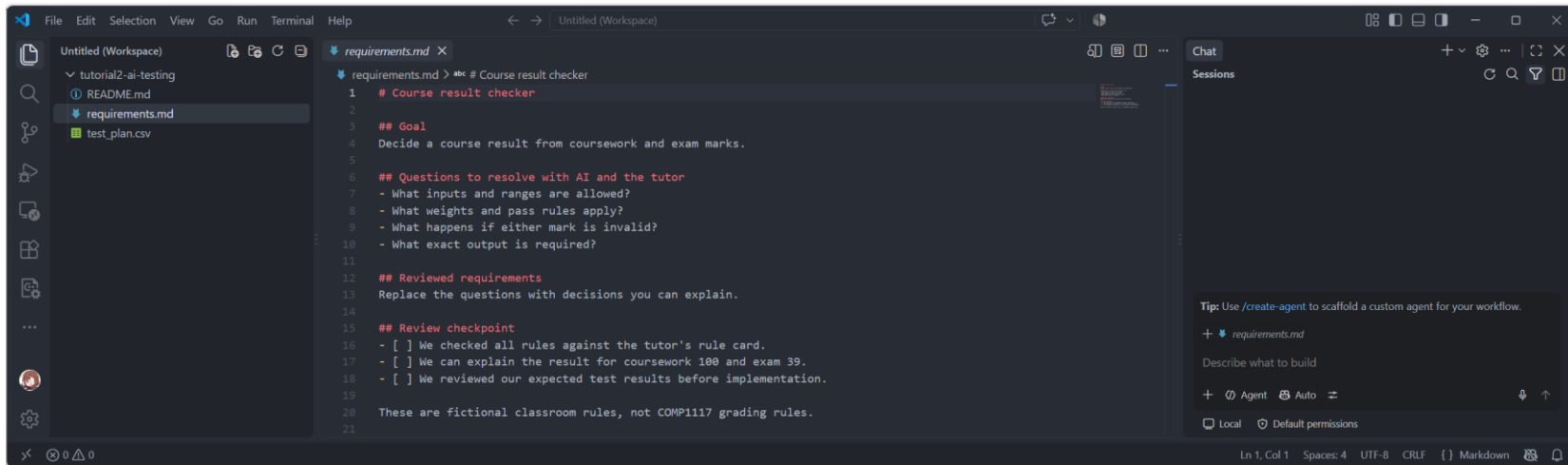
A student wants a small program to decide a course result from coursework and exam marks.

- ▶ The result depends on weighted marks and an exam minimum.
- ▶ The program must recognise marks outside the allowed range.
- ▶ We need to agree the exact rules before writing Python.

All grading rules in this exercise are fictional.

A workspace for the tutorial

Create tutorial2-ai-testing, then open it using File > Open Folder.



Save all files for this task in the same folder.

Start requirements.md

```
# Course result checker
```

```
## Goal
```

```
Decide a result from coursework and exam marks.
```

```
## Questions to resolve
```

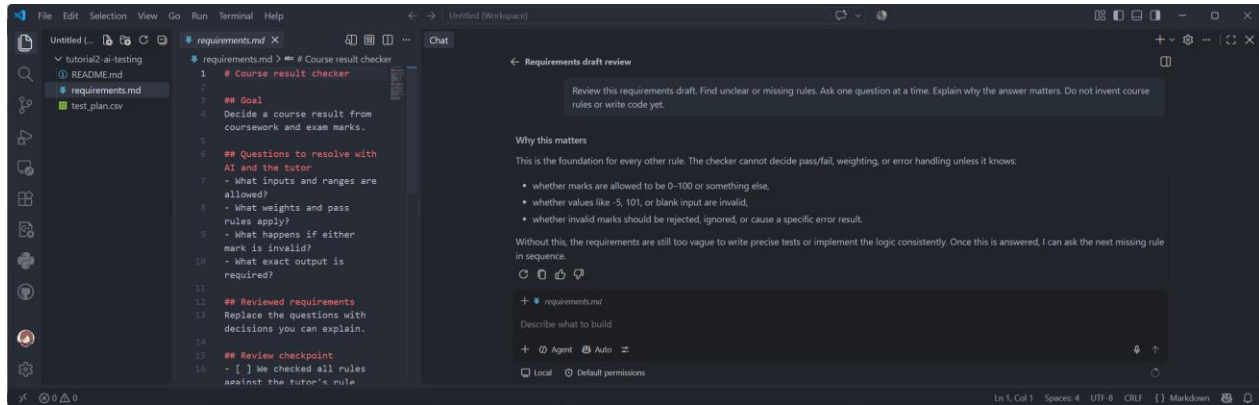
- What inputs and ranges are allowed?
- What weights and pass rules apply?
- What should happen for an invalid mark?
- What exact output is required?

This file records the decisions that your code must follow.

AI as a requirements reviewer

Attach requirements.md using Add Context or a # file mention.

Ask: “Find unclear requirements. Ask one question at a time. Do not invent course rules or write code yet.”



Answer from the brief. Ask why a missing detail matters.

The tutor's rule card

Topic	Agreed rule for this exercise
Inputs	Coursework first, exam second. Both are integers.
Valid range	Each mark is from 0 to 100 inclusive.
Weighted total	40% coursework + 60% exam. Do not round.
Pass decision	Total is at least 50 AND exam is at least 40.
Invalid marks	If either mark is out of range, print Invalid.
Output / scope	Print only Pass, Fail or Invalid. One pair per run.

Use these fictional rules to answer the AI's questions.

Reviewed requirements.md

```
# Course result checker
```

```
R1: Read coursework, then exam, as integers.
```

```
R2: Validate both marks: 0 to 100 inclusive.
```

```
R3: total = 0.4 * coursework + 0.6 * exam.
```

```
R4: Pass needs total >= 50 AND exam >= 40.
```

```
R5: Print Invalid, Pass or Fail only.
```

```
R6: Handle one pair of integer inputs per run.
```

Checkpoint: explain Invalid, Fail and Pass without looking at code.

Predict outputs before coding

Coursework	Exam	Your prediction
50	50	Pass or Fail? Why?
100	39	Pass or Fail? Why?
80	101	Valid or Invalid? Why?

Derive the answer from requirements.md before asking AI.

What is a test case?

A test records the inputs and the result required by the specification.

Coursework	Exam	Expected	Actual
100	39	Fail	Fill in after running

Purpose: checks the exam minimum when the weighted total is high.

Test groups and boundaries

What could go wrong?	Useful tests
Input validation	Check each mark at 0 and 100, and just outside.
Weighted threshold	Keep exam at 50. Try coursework 49, 50 and 51.
Exam minimum	Keep coursework at 100. Try exam 39, 40 and 41.
Ordinary behaviour	Try a clear pass and a clear fail.

Your test groups should cover every reachable if/elif/else branch.

Test groups and branch coverage

After implementation, check that your tests exercise both True and False outcomes of each if/elif decision.

Example mapping for the reviewed implementation:

Branch taken	Coursework / Exam	Expected
if: a mark is outside 0–100	80 / 101	Invalid
elif: both pass conditions hold	80 / 80	Pass
else: valid marks fail a pass rule	100 / 39	Fail

An uncovered branch means a missing test group or case.

Branch coverage is a minimum check. Keep boundary and separate-input tests.

Reference tests and their purpose

Coursework	Exam	Expected	Purpose
80	80	Pass	Ordinary passing case
49	50	Fail	Weighted total just below 50
50	50	Pass	Weighted total exactly 50
100	39	Fail	Exam just below its minimum
100	40	Pass	Exam exactly at its minimum
-1	80	Invalid	Coursework below its range
80	101	Invalid	Exam above its range

This is a starting set. Add the remaining input-range edges.

Implementation after the review

Attach requirements.md and test_plan.csv.

“We have reviewed these requirements and expected results.

Explain your plan and wait for my review. After approval, create result_checker.py using beginner-level Python. Keep the agreed requirements and test expectations.

Run the cases and report the command, actual output and any mismatch. Explain every change you make.”

Review the plan and code. Keep the required evidence.

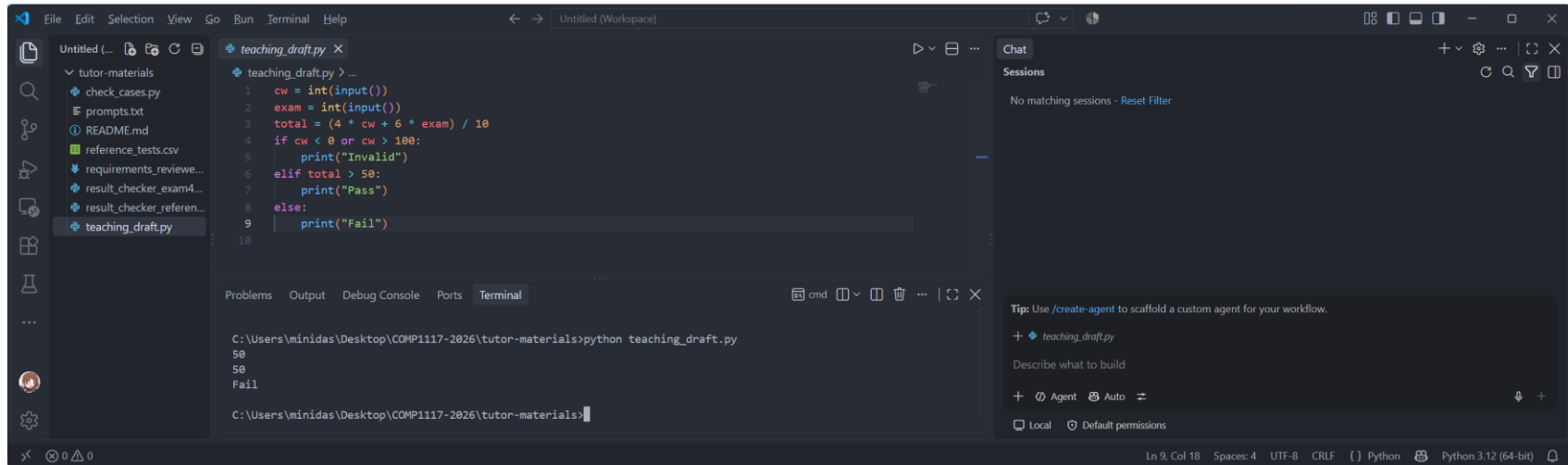
A teaching draft with bugs

Use this deliberately faulty draft to practise verification.

```
cw = int(input())
exam = int(input())
total = (4 * cw + 6 * exam) / 10
if cw < 0 or cw > 100:
    print("Invalid")
elif total > 50:
    print("Pass")
else:
    print("Fail")
```

Run tests and record evidence

Run the program again for each input pair. Record what it prints.



The screenshot shows a Visual Studio Code editor window with a dark theme. The left sidebar displays a file explorer with a project named 'tutor-materials' containing files like 'check_cases.py', 'prompts.txt', 'README.md', 'reference_tests.csv', and 'teaching_draft.py'. The main editor area shows the code for 'teaching_draft.py', which takes two inputs, calculates a total, and prints 'Invalid', 'Pass', or 'Fail' based on the total. The bottom panel shows the 'Terminal' tab with the command 'python teaching_draft.py' and its output: '50', '50', and 'Fail'. The status bar at the bottom indicates the file is at line 9, column 18, using UTF-8 encoding with CRLF line endings, and is a Python 3.12 (64-bit) file.

```
File Edit Selection View Go Run Terminal Help
Untitled (Workspace)
teaching_draft.py x
teaching_draft.py > ...
1 cw = int(input())
2 exam = int(input())
3 total = (4 * cw + 6 * exam) / 10
4 if cw < 0 or cw > 100:
5     print("Invalid")
6 elif total > 50:
7     print("Pass")
8 else:
9     print("Fail")
10

Problems Output Debug Console Ports Terminal
C:\Users\minidas\Desktop\COMP1117-2026\tutor-materials>python teaching_draft.py
50
50
Fail
C:\Users\minidas\Desktop\COMP1117-2026\tutor-materials>

Ln 9, Col 18 Spaces: 4 UTF-8 CRLF {} Python Python 3.12 (64-bit)
```

Expected: Pass. Actual from this draft: Fail.

What the tests reveal

Coursework / Exam	Expected	Draft output	Verdict
80 / 80	Pass	Pass	Passed
50 / 50	Pass	Fail	FAILED
100 / 39	Fail	Pass	FAILED
80 / 101	Invalid	Pass	FAILED

A passed ordinary test can coexist with several broken rules.

A bug report tied to a requirement

Field	Example
File and inputs	teaching_draft.py: coursework 100, exam 39
Expected / actual	Expected Fail, but the program printed Pass.
Requirement	R4 requires the exam mark to be at least 40.
Why this case helps	Total is 63.4, so it isolates the exam-minimum rule.

Describe the failing rule before suggesting a code change.

Repair with AI, then retest

“For coursework 100 and exam 39, the program printed Pass. R4 requires Fail because the exam minimum is 40.

Explain the cause and propose a small fix. Keep the expected results unchanged. After the fix, rerun the full test plan and show the actual outputs.”

Inspect the change and the run results before accepting the repair.

A reviewed implementation

```
cw = int(input())
exam = int(input())
if cw < 0 or cw > 100 or exam < 0 or exam > 100:
    print("Invalid")
elif exam >= 40 and 4 * cw + 6 * exam >= 500:
    print("Pass")
else:
    print("Fail")
```

$4 \times \text{coursework} + 6 \times \text{exam} \geq 500$ means a weighted total ≥ 50 .

Regression and missing cases

Retest	Reason
Original test plan	Check behaviour that worked before the repair.
Other input-range edges	Try 0, 100 and the missing out-of-range side.
Ordinary fail: 20 / 20	Check ordinary behaviour as well as failures.
Every test that exposed a bug	Detect the same fault if it returns.

A code fix keeps the agreed requirements and expected answers.

Independent check: a changed rule

4 minutes: predict first, then discuss with AI.

Change only the minimum exam mark from 40 to 45.

Keep the weighting and the weighted pass threshold unchanged.

- ▶ Update R4 in requirements.md.
- ▶ Design a pair of tests at the new exam boundary.
- ▶ Identify an old expected answer that must now change.

Compare your new tests

Coursework	Exam	New expected	Reason
100	44	Fail	Just below the new exam minimum
100	45	Pass	Exactly at the new minimum
100	40	Fail	An old Pass now changes to Fail
50	50	Pass	Weighted threshold is unchanged

Change expected answers when the requirement changes, and explain why.

Exit check and evidence

- ▶ Which requirement did AI help you clarify?
- ▶ Why can 100 / 39 expose a bug that 80 / 80 misses?
- ▶ Which actual output shows that your repair worked?
- ▶ Can you explain the final code without asking AI?

Keep requirements.md, the test plan, code and the required AI-use record.

References and setup links

[HKU GenAI access](#)

[HKU Copilot setup](#)

[VS Code Chat](#)

[Choosing a chat model](#)

[Adding files as context](#)

[Test design: ISTQB syllabus](#)

[Python control flow](#)